MCMP117 · Week 14 · Course Notes

Fundamentals of Operating Systems

Introduction to Shell Scripting

Type it once, save it, run it a thousand times.

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain what a shell script is, why it is useful, and what happens (fork, exec, shebang) when you run one.
2. Write a script with a shebang line, make it executable with `chmod +x` and run it in the four possible ways.
3. Use variables, the three kinds of quoting, command substitution `$( )` and integer arithmetic `$(( ))`.
4. Read arguments (`$0 $1 $# $@`) and keyboard input (`read`).
5. Use exit statuses, `test` / `[ ]` for numbers, strings and files, and `if / elif / else`.
6. Write `for` and `while` loops and simple functions.
7. Debug a script with `bash -n` and `bash -x`, and recognise the most common error messages.
8. Read and adapt two complete automation scripts: `backup.sh` and `report.sh`.

Contents

1 From Week 13 to Week 14 1

2 Why Write Scripts? 2

3 Your First Script 2

4 Variables, Quoting and Input 4

5 Exit Status and Tests 7

6 Making Decisions: `if` 9

7 Loops 11

8 Syntax Reference 14

9 Two Real Automation Scripts 14

10 Common Errors and What They Mean 17

11 Summary 19

12 Glossary 19

13 Try It Yourself 20

14 Review Questions 20

15 Looking Ahead: Week 15 21

16 Lab Sheet: Graded Script-Writing Exercises 21

17 Lab Sheet Solutions 23

1 From Week 13 to Week 14

Last week we looked at **processes** from the command line: every command runs as a process with a PID and a parent, `ps` and `top` show them, jobs can be moved between foreground and background, and `kill` sends signals. In Weeks 9–12 we learned the commands themselves: navigation, file operations, permissions, redirection and pipelines.

So far we have typed every command by hand. This week we save commands in a file and let the shell run them for us. Such a file is a **shell script**. A script is just another program: when you run it, the shell creates a new process for it, exactly as for `ls` or `sleep`, and everything from Week 13 applies to it.

How to follow these notes. All example scripts are in the `code/` folder next to these notes. Create a fresh lab folder, copy the scripts and work in `~/lab/scripts`:

```
$ bash mcmp117_lab_setup.sh          # re-creates ~/lab (same files for everybody)
$ cp *.sh ~/lab/scripts/             # run this in the week14/code folder
$ cd ~/lab/scripts
```

Every output in these notes was captured as the ordinary user `student` on our test machine `okan-lab`, with `LC_ALL=C.UTF-8`. On your machine, dates, times, file sizes, PIDs and user names will differ, and the order of `ls` or `sort` output can differ with the locale. The shell is GNU `bash` 5.2.

2 Why Write Scripts?

Imagine that every morning you back up a folder, check a log file for errors and count yesterday's web requests: six or seven commands, typed in the right order, with the right paths. Typing them by hand is slow, easy to get wrong, hard to explain to a colleague, and impossible while you are asleep. A script solves all four problems:

- **Repeatability:** one command, `./backup.sh`, does the same steps every time.
- **Fewer mistakes:** a typo is fixed once in the file and stays fixed.
- **Documentation:** the script, with comments, *is* the description of the procedure.
- **Automation:** a scheduler such as `cron` can start a script at fixed times without anybody at the keyboard.

A useful rule of thumb: if you type the same sequence of commands for the third time, write a script. System administrators, DevOps engineers and embedded-Linux developers write small scripts every day; Linux itself runs many shell scripts while it boots.

3 Your First Script

3.1 Anatomy of a script

A script is a plain text file. Create it with any text editor, for example `nano hello.sh` (save with Ctrl+O and Enter, exit with Ctrl+X):

```
#!/bin/bash
# hello.sh -- my first shell script
# Run: bash hello.sh   or: chmod +x hello.sh; ./hello.sh
echo "Hello from a shell script!"
echo "I am running as: $(whoami)"
echo "Current folder:  $(pwd)"
```

- **Line 1, the shebang:** `#!` followed by the absolute path of the program that must run the file. `#!/bin/bash` chooses bash; `#!/usr/bin/python3` would run a Python program in exactly the same way. It is special *only* on the very first line.
- **Comments:** everything after a # is ignored by bash and is written for humans.
- **Commands:** exactly what you would type at the prompt. `$(whoami)` is *command substitution* (Section 4.2).

The ending `.sh` is only a convention for humans. Linux does not look at extensions (Week 7: file types); it decides by the execute permission and the first line.

3.2 Running it

```
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello from a shell script!
I am running as: student
```

```
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ echo $?
126
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ chmod +x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrwxr-x 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
```

What happened:

- `bash hello.sh` starts bash and gives it the file name as an argument. Bash only *reads* the file, so read permission is enough.
- `./hello.sh` asks the kernel to *execute* the file, which requires the x permission. The file was created with mode `-rw-rw-r--` (the default umask 002 of Week 11 gives $666 - 002 = 664$), so the result is `Permission denied` and the exit status is 126 ("found, but cannot be executed").
- The command `chmod +x hello.sh` adds the x permission for user, group and others: $664 \to 775$, i.e. `rwxrwxr-x`. Now `./hello.sh` works.
- `hello.sh` without `./` is *not found*: the shell searches for commands only in the folders listed in the variable PATH (try `echo $PATH`), and the current folder is deliberately not one of them. `./` means "the file in *this* folder".

3.3 What happens under the hood

Running `./hello.sh` uses the mechanisms of Weeks 3, 11 and 13:

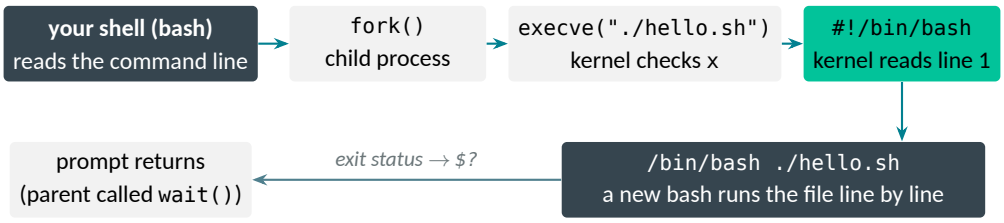


Figure 1: Running a script: fork, exec, shebang, a new interpreter process, and the exit status back to the parent.

The shell `fork()`s a child (Week 3), and the child calls `execve()` on the file. The kernel checks the execute permission (Week 11). A text file is not machine code, so the kernel looks at its first two bytes: `#!` means "start the program named on this line and pass it the script's path". The kernel therefore actually runs `/bin/bash ./hello.sh`. The parent shell waits; when the script ends, its exit status is returned to the parent and is available as `$?`.

A script runs in a child process

Variables you set and `cd` commands you run *inside* a script change only the child process. When the script ends, your terminal is exactly where it was. (To run a file inside the *current* shell, use `source file` or `. file`; this is how `~/.bashrc` is read.)

3.4 Four ways to run a script

Command	Needs	Runs in	Typical use
`bash hello.sh`	r	a new bash process	quick test, no x bit
`./hello.sh`	r+x, shebang	a new process	normal way in this course
`hello.sh`	r+x, folder in PATH	a new process	installed tools (e.g. in `~/bin`)
`source hello.sh`	r	the *current* shell	files that set variables

4 Variables, Quoting and Input

4.1 Variables

A variable is created by assignment and used with a `$`:

```
student@okan-lab:~/lab/scripts$ name="Ayse"
student@okan-lab:~/lab/scripts$ echo $name
Ayse
student@okan-lab:~/lab/scripts$ echo "Hello, $name!"
Hello, Ayse!
student@okan-lab:~/lab/scripts$ name = Ayse
bash: name: command not found
```

The rules are short but strict:

- **No spaces around =.** Bash separates words at spaces, so `name = Ayse` is the command `name` with the two arguments `=` and `Ayse`, hence `name: command not found`. (On Ubuntu the message is longer because of the *command-not-found* helper; the meaning is the same.)
- Names consist of letters, digits and `_`, do not start with a digit, and are case-sensitive (`name` and `NAME` are different).
- `${name}` is the long form; it is needed when letters follow the name: `"${name}s"`.
- Every value is text. `count=5` stores the text “5”, not a number; arithmetic needs `$(( ))`.
- An unset variable is simply empty; bash does not complain.
- Convention: UPPERCASE names for environment variables (`HOME`, `USER`, `PATH`, `SHELL`), lowercase for your own.

4.2 Command substitution and arithmetic

```
#!/bin/bash
# variables.sh -- variables, command substitution and arithmetic
name="Ayse"                    # NO spaces around the = sign
```

```
course=MCMP117
week=14
echo "Student: $name, course: $course, week $week"

today=$(date +%Y-%m-%d)          # $( ) = the OUTPUT of a command
lines=$(wc -l < ~/lab/docs/syllabus.txt)
echo "Today is $today; the syllabus has $lines lines"

left=$((15 - week))              # $(( )) = integer arithmetic
echo "Weeks left until the final exam: $left"
echo "7 / 2 = $((7 / 2)) remainder $((7 % 2))   (integers only!)"
```

```
student@okan-lab:~/lab/scripts$ ./variables.sh
Student: Ayse, course: MCMP117, week 14
Today is 2026-10-12; the syllabus has 15 lines
Weeks left until the final exam: 1
7 / 2 = 3 remainder 1   (integers only!)
```

- **Command substitution** `$(command)` runs the command and inserts its output (without the final newline). The old form with backquotes does the same but is harder to read and to nest. Note `wc -l < file`: with input redirection (Week 12) wc prints only the number, without the file name — exactly what we want inside `$( )`.
- **Arithmetic expansion** `$((expression))` computes with integers: `+ - * / %` and parentheses, with the usual precedence: $3 + 4 \times 2 = 11$ (see the transcript above). Inside, the `$` before variable names is optional.
- **Integers only:** $7/2$ gives 3 and 7 mod $2 = 1$, because bash uses 64-bit integer arithmetic. For decimals use awk (as in `report.sh`) or the calculator `bc`.

4.3 Quoting

```
#!/bin/bash
# quotes.sh -- double quotes, single quotes, backslash, no quotes
name="Ayse"
file="final report.txt"          # a file name that contains a space
# shellcheck disable=SC2016      # (on purpose)
echo 'Single: Hello $name'       # '...' : everything is literal
echo "Double: Hello $name"       # "..." : $var and $( ) expanded
echo "Escaped: it costs \$5"     # \$ : one literal dollar sign
cd ~/lab/docs || exit 1
wc -l "$file"                    # quoted:   ONE argument
# shellcheck disable=SC2086      # (on purpose)
wc -l $file                      # unquoted: TWO arguments
```

```
student@okan-lab:~/lab/scripts$ ./quotes.sh
Single: Hello $name
Double: Hello Ayse
Escaped: it costs $5
2 final report.txt
wc: final: No such file or directory
wc: report.txt: No such file or directory
0 total
```

Form	`$var`, `$( )` expanded?	Spaces kept?
`"double quotes"`	yes	yes — one word
`'single quotes'`	no, everything is literal	yes — one word
backslash `\$`	makes *one* character literal	—
no quotes	yes	no — split into words, * expanded

The last two lines of `quotes.sh` show the most common bug in real scripts. The variable `file` contains a space. Quoted, `"$file"` is one argument and `wc` counts 2 lines. Unquoted, the value is split into the two arguments `final` and `report.txt`, and `wc` reports two missing files. **Rule: put double quotes around every `$variable` and every `$( )`**, unless you have a reason not to.

The two `# shellcheck disable` lines are for **ShellCheck**, a free tool (`https://www.shellcheck.net`, or `sudo apt install shellcheck`) that warns about exactly these problems. They tell it that the mistakes are intentional. All other example scripts of this week pass ShellCheck without warnings.

4.4 Positional parameters

The words typed after the script name are its **arguments**. Inside the script they are $1, $2, ...:

```
#!/bin/bash
# args.sh -- positional parameters: the words typed after the script name
echo "Script name  \$0 = $0"
echo "First arg    \$1 = $1"
echo "Second arg   \$2 = $2"
echo "Arg count    \$# = $#"
echo "All args     \$@ =" "$@"
i=1
for arg in "$@"; do             # "green tea" stays ONE word
    echo "  argument $i: [$arg]"
    i=$((i + 1))
done
```

```
student@okan-lab:~/lab/scripts$ ./args.sh apple "green tea" 42
Script name  $0 = ./args.sh
First arg    $1 = apple
Second arg   $2 = green tea
Arg count    $# = 3
All args     $@ = apple green tea 42
  argument 1: [apple]
  argument 2: [green tea]
  argument 3: [42]
student@okan-lab:~/lab/scripts$ ./args.sh
Script name  $0 = ./args.sh
First arg    $1 =
Second arg   $2 =
Arg count    $# = 0
All args     $@ =
student@okan-lab:~/lab/scripts$ ./greet.sh
What is your name? Ayse
Which year were you born? 2006
Nice to meet you, Ayse. You are about 20 years old.
```

Variable	Meaning
`$0`	the name of the script, as it was typed
`$1` … `$9`	the 1st … 9th argument (`${10}` for the 10th)
`$#`	the number of arguments
`"$@"`	all arguments, each kept as a separate word (use in loops)
`$*`	all arguments glued into one string
`$?`	exit status of the last command (Section 5)
`$$`	PID of the current shell (Week 13)

The shell splits the command line into words *before* the script starts, so `"green tea"` arrives as one argument: `$#` is 3, not 4. Without arguments, `$1` is empty and `$#` is 0; this is why well-written scripts check `$#` first. The transcript above also contains the last demo of this subsection, `read`.

4.5 Keyboard input with `read`

```
#!/bin/bash
# greet.sh -- reading input from the keyboard with read
read -r -p "What is your name? " name
read -r -p "Which year were you born? " year
age=$(( $(date +%Y) - year ))
echo "Nice to meet you, $name. You are about $age years old."
```

In the transcript the user typed `Ayse` and `2006`; the age is $2026 - 2006 = 20$. `read -r var` waits for one line of input and stores it; `-p` prints a prompt first; `-r` keeps backslashes as typed (always use it). With several names, `read -r first last` splits the line at spaces.

Arguments or `read`? Arguments are better for automation: `./grade.sh 72` can be called from another script or from `cron` with nobody at the keyboard. `read` is for interactive tools such as menus and games.

5 Exit Status and Tests

5.1 Every command answers yes or no

When a process ends, it returns a small number, its **exit status**, to its parent (in C: `return n` from `main` or `exit(n)`, Week 3; the parent collects it with `wait()`). The shell stores it in `$?`:

```
student@okan-lab:~/lab/scripts$ ls ../docs > /dev/null
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ ls ../nothing
ls: cannot access '../nothing': No such file or directory
student@okan-lab:~/lab/scripts$ echo $?
2
student@okan-lab:~/lab/scripts$ nosuchcommand
bash: nosuchcommand: command not found
student@okan-lab:~/lab/scripts$ echo $?
127
student@okan-lab:~/lab/scripts$ grep -q ERROR ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ grep -q PANIC ../logs/system.log
```

```
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ [ 5 -gt 3 ]
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ [ "abc" = "abd" ]
student@okan-lab:~/lab/scripts$ echo $?
1
```

`$?`	**Conventional meaning**
0	success, “true”
1	general failure, “false” (e.g. `grep`: not found)
2	wrong usage or serious error (e.g. `ls` of a missing file)
126	command found but not executable (Permission denied)
127	command not found

0 means success

In the shell, 0 is *success* (“true”) and anything from 1 to 255 is failure. This is the opposite of C, where 0 is false. It makes sense: there is one way to succeed, but many ways to fail, and the number can say which.

`$?` always refers to the *last* command, so look at it immediately — the `echo` itself sets it to 0 again. `grep -q` prints nothing at all; it only answers with its exit status, which makes it ideal for decisions.

5.2 `test` and `[ ]`

`test` compares numbers, strings or files and answers with its exit status. `[` is another name for `test` whose last argument must be `]`. The transcript shows that `[` is a shell builtin — a *command*. This single fact explains every spacing rule: in `[ "$n" -gt 3 ]` the five words `[`, `"$n"`, `-gt`, `3`, `]` are separate arguments of the command `[`, so they must be separated by spaces. Writing `[$n` produces the word `[5`, which is not a command.

Type	**Operator**	**True when … (example)**
Numbers	`-eq -ne`	equal / not equal: `[ "$n" -eq 0 ]`
	`-lt -le`	less than / less or equal: `[ "$x" -lt 10 ]`
	`-gt -ge`	greater than / greater or equal: `[ "$s" -ge 50 ]`
Strings	`= !=`	same text / different text: `[ "$a" = yes ]`
	`-z -n`	string is empty / not empty: `[ -z "$1" ]`
Files	`-e`	path exists (any type): `[ -e "$p" ]`
	`-f -d`	regular file / directory: `[ -d ~/lab ]`
	`-r -w -x`	readable / writable / executable by you
	`-s`	file exists and is not empty
Logic	`!`	negation: `[ ! -d "$dir" ]`
	`&& ||`	and / or between two tests: `[ -f a ] && [ -f b ]`

Two traps: numbers are compared with *letters* (`-lt`, `-gt`), strings with symbols (=, !=). Inside `[ ]`, < and > are redirections (Week 12), not comparisons. And `-eq` compares values while = compares text: `[ 05 -eq 5 ]` is true, `[ 05 = 5 ]` is false. (Bash also offers `[[ … ]]`, a more forgiving keyword; we use the portable `[ ]`.)

6 Making Decisions: `if`

6.1 The shape of an `if`

```
if TEST; then
    commands            # run when TEST succeeds (exit status 0)
elif OTHER_TEST; then
    commands            # optional, as many as you like
else
    commands            # optional: when nothing above succeeded
fi                      # "if" backwards ends the block
```

TEST can be *any* command: `[ … ]`, `grep -q`, `mkdir`, …. The semicolon before `then` is needed because `then` must start a new command; writing `then` on the next line works as well. Bash tries the tests from top to bottom and runs only the first branch whose test succeeds.

6.2 Example: `check.sh`

```
#!/bin/bash
# check.sh -- file tests ([ -f ], [ -d ], [ -e ]) and exit statuses
if [ $# -ne 1 ]; then                   # exactly one argument?
    echo "Usage: $0 PATH" >&2           # >&2 : message goes to stderr
    exit 2                              # 2 = wrong usage
fi
path=$1
if [ -f "$path" ]; then
    echo "$path: regular file, $(wc -l < "$path") lines"
elif [ -d "$path" ]; then
    echo "$path: directory"
elif [ -e "$path" ]; then
    echo "$path: exists, but is neither a file nor a directory"
else
    echo "$path: does not exist" >&2
    exit 1                              # 1 = failure
fi
exit 0                                  # 0 = success
```

```
student@okan-lab:~/lab/scripts$ ./check.sh ../docs/syllabus.txt
../docs/syllabus.txt: regular file, 15 lines
student@okan-lab:~/lab/scripts$ ./check.sh ../logs
../logs: directory
student@okan-lab:~/lab/scripts$ ./check.sh /dev/null
/dev/null: exists, but is neither a file nor a directory
student@okan-lab:~/lab/scripts$ ./check.sh nothing.txt
nothing.txt: does not exist
student@okan-lab:~/lab/scripts$ echo $?
1
```

```
student@okan-lab:~/lab/scripts$ ./check.sh
Usage: ./check.sh PATH
student@okan-lab:~/lab/scripts$ echo $?
2
```

Each kind of path takes a different branch. `/dev/null` exists (`-e`) but is a *device file* (Week 9: `/dev`), so it is neither `-f` nor `-d`. The script also shows three good habits: it checks `$#` and prints a *Usage* line; it sends error messages to standard error with >&2 (Week 12), so that `./check.sh x > out.txt` still shows the error on the screen; and it uses distinct exit codes (0 success, 1 failure, 2 wrong usage) that a calling script can test.

6.3 Numeric tests: `grade.sh`

```
#!/bin/bash
# grade.sh -- numeric tests with if / elif / else
# Usage: ./grade.sh SCORE        (an example scale, not an official one)
score=$1
if [ -z "$score" ]; then               # -z : the string is empty
    echo "Usage: $0 SCORE (0-100)" >&2
    exit 2
fi
if [ "$score" -lt 0 ] || [ "$score" -gt 100 ]; then
    echo "Error: score must be between 0 and 100" >&2
    exit 1
elif [ "$score" -ge 85 ]; then
    letter=A
elif [ "$score" -ge 70 ]; then
    letter=B
elif [ "$score" -ge 50 ]; then
    letter=C
else
    letter=F
fi
echo "Score $score -> grade $letter"
```

```
student@okan-lab:~/lab/scripts$ ./grade.sh 92
Score 92 -> grade A
student@okan-lab:~/lab/scripts$ ./grade.sh 72
Score 72 -> grade B
student@okan-lab:~/lab/scripts$ ./grade.sh 49
Score 49 -> grade F
student@okan-lab:~/lab/scripts$ ./grade.sh 150
Error: score must be between 0 and 100
student@okan-lab:~/lab/scripts$ ./grade.sh
Usage: ./grade.sh SCORE (0-100)
```

The `elif` tests form a staircase, and their order matters: if `-ge 50` came first, a score of 92 would get a C. `||` means “or”: the second test runs only if the first one failed. (The scale is only an example, not an official grading scale.)

6.4 Debugging

Three tools find most bugs:

- `bash -n script.sh` checks the syntax without running anything (Section 10).
- `bash -x script.sh` prints every command, marked with +, *after* expansion and before it runs. Inside a script, `set -x` … `set +x` traces only a part.
- Reading the error message: it names the script, the *line number* and the word bash could not handle.

```
student@okan-lab:~/lab/scripts$ ./grade.sh abc
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 12: [: abc: integer expression expected
./grade.sh: line 14: [: abc: integer expression expected
./grade.sh: line 16: [: abc: integer expression expected
Score abc -> grade F
student@okan-lab:~/lab/scripts$ bash -x grade.sh 72
+ score=72
+ '[' -z 72 ']'
+ '[' 72 -lt 0 ']'
+ '[' 72 -gt 100 ']'
+ '[' 72 -ge 85 ']'
+ '[' 72 -ge 70 ']'
+ letter=B
+ echo 'Score 72 -> grade B'
Score 72 -> grade B
```

The trace shows that `$score` became 72, that the test `-ge 85` failed and `-ge 70` succeeded. The run with `abc` reveals a real bug: every numeric test complains `integer expression expected` and counts as false, so the script falls through to `else` and prints grade F. (Line 9 appears twice because both tests joined by `||` are tried.) The fix is to validate input before using it, for example

```
if ! echo "$score" | grep -qx '[0-9]\{1,3\}'; then
    echo "Error: not a number: $score" >&2; exit 1
fi
```

where `grep -x` requires the whole line to match one to three digits (Week 12 regular expressions).

7 Loops

7.1 `for`: once for every item

```
#!/bin/bash
# loops.sh -- three kinds of for loop
for city in Istanbul Ankara Izmir; do       # 1) over a list of words
    echo "City: $city"
done

for i in {1..5}; do                          # 2) over a range of numbers
    echo -n "$i squared is $((i * i)); "
done
echo                                         # finish the line
```

```
11
12 for f in ~/lab/docs/*.txt; do               # 3) over files (wildcard)
13     echo "$(basename "$f"): $(wc -l < "$f") lines"
14 done
```

```
student@okan-lab:~/lab/scripts$ ./loops.sh
City: Istanbul
City: Ankara
City: Izmir
1 squared is 1; 2 squared is 4; 3 squared is 9; 4 squared is 16; 5 squared is 25;
final report.txt: 2 lines
linux_quotes.txt: 5 lines
notes_01.txt: 1 lines
notes_02.txt: 1 lines
notes_03.txt: 1 lines
syllabus.txt: 15 lines
```

The general form is `for VAR in LIST; do …; done`. The list can be:

- words written out (`Istanbul Ankara Izmir`) or the arguments `"$@"`;
- a brace expansion `{1..5}` → `1 2 3 4 5` (Week 10). Braces cannot contain variables: `{1..$n}` does not work; use `$(seq 1 "$n")` or a `while` loop instead;
- a wildcard such as `~/lab/docs/*.txt`: the shell replaces it by the matching file names. Because the loop variable is quoted (`"$f"`), the name `final report.txt` stays in one piece;
- the output of a command, `$(command)` (split at spaces and newlines).

The output also shows `1 lines`: a script prints exactly what it is told; printing “1 line” would need an `if`. Bash also has a C-style loop, `for ((i = 1; i <= 5; i++))`, used in Lab exercise 16. A wildcard works directly at the prompt too:

```
student@okan-lab:~/lab/scripts$ for f in ../photos/*.png ../photos/*.JPG; do echo "found $f";
    done
found ../photos/img013.png
found ../photos/IMG_014.JPG
```

7.2 `while`: as long as a test succeeds

```
1  #!/bin/bash
2  # while.sh -- a counting loop, and a file read line by line
3  n=3
4  while [ "$n" -gt 0 ]; do                    # repeat WHILE the test is true
5      echo "Countdown: $n"
6      n=$((n - 1))
7  done
8  echo "Go!"
9
10 sum=0
11 count=0
12 while read -r x; do                         # one line of the file per turn
13     sum=$((sum + x))
14     count=$((count + 1))
```

```
done < ~/lab/data/numbers.txt               # the whole loop reads this file
echo "$count numbers, sum = $sum, average = $((sum / count)) (integer)"
```

```
student@okan-lab:~/lab/scripts$ ./while.sh
Countdown: 3
Countdown: 2
Countdown: 1
Go!
20 numbers, sum = 11325, average = 566 (integer)
```

Two patterns cover most uses. A *counter* loop changes its variable in every turn; forget `n=$((n - 1))` and the loop never ends (stop it with Ctrl+C, i.e. SIGINT, Week 13). A *read loop*, `while read -r x; do …; done < file`, reads one line per turn because the redirection `< file` after `done` feeds the whole loop; at the end of the file `read` fails and the loop stops. The numbers check out: the 20 values sum to $11\,325$, and $11\,325/20 = 566.25$, which integer arithmetic truncates to 566. (`until TEST; do …; done` loops while the test *fails*.)

7.3 Functions

```
#!/bin/bash
# functions.sh -- define a block of commands once, call it many times
line() {                                    # no arguments
    echo "------------------------------------"
}
count_word() {                              # $1 = word, $2 = file
    local n                                 # local: exists only in here
    n=$(grep -c -- "$1" "$2")
    echo "$1 appears on $n lines of $(basename "$2")"
}
is_even() {                                 # the exit status is the answer
    [ $(( $1 % 2 )) -eq 0 ]
}
line
count_word ERROR ~/lab/logs/system.log
count_word WARN  ~/lab/logs/system.log
line
for k in 7 12; do
    if is_even "$k"; then echo "$k is even"; else echo "$k is odd"; fi
done
```

```
student@okan-lab:~/lab/scripts$ ./functions.sh
----------------------------------
ERROR appears on 13 lines of system.log
WARN appears on 28 lines of system.log
----------------------------------
7 is odd
12 is even
```

A function is a named block of commands, defined before use with `name() { …; }` and called like a command. Inside it, `$1`, `$2`, … are the *function's* arguments, not the script's. `local` keeps a variable private to the function. A function's exit status is that of its last command (or set with `return n`, 0–255), so `is_even` can be used directly in an `if`. To give back *text*, `echo` it and capture it with `$( )`. Why `local`

n on its own line? `local n=$(grep …)` would hide `grep`'s exit status behind that of `local`.

8 Syntax Reference

Structure	Syntax	Remember
shebang	`#!/bin/bash`	line 1 only
variable	`name=value`, `"$name"`, `"${name}s"`	no spaces at =
command subst.	`today=$(date +%F)`	output of a command
arithmetic	`n=$((n + 1))`, `$((a % b))`	integers only
arguments	`$0 $1 ... $9 $# "$@"`	check $#
input	`read -r -p "Name? " name`	`-r` always
if	`if [ "$n" -gt 0 ]; then ...; elif ...; then ...; else ...; fi`	spaces in `[ ]`
for (list)	`for c in a b c; do echo "$c"; done`	
for (range)	`for i in {1..5}; do ...; done`	no variables in {}
for (files)	`for f in *.txt; do wc -l "$f"; done`	quote `"$f"`
for (C style)	`for ((i = 0; i < 5; i++)); do ...; done`	bash only
while	`while [ "$n" -gt 0 ]; do n=$((n - 1)); done`	change the variable
read a file	`while read -r line; do ...; done < file`	< after done
function	`name() { local x=$1; echo "$x"; }`	define before use
and / or	`mkdir -p d && echo ok; [ -f f ] || exit 1`	short `if`
exit	`exit 0` (success), `exit 1` (failure)	0–255
debugging	`bash -n s.sh` (syntax), `bash -x s.sh` (trace)	

Two handy expansions (optional)

`${var:-default}` gives the value of `var`, or `default` if it is empty or unset: `n=${1:-7}` means "the first argument, or 7". Lab exercises 7, 8 and 14 use it. `${#var}` gives the length of the string in `var`.

9 Two Real Automation Scripts

9.1 `backup.sh`: a timestamped archive

```
#!/bin/bash
# backup.sh -- timestamped .tar.gz backup of ~/lab/docs into ~/lab/backup
# Usage: ./backup.sh            (works from any folder)
lab="$HOME/lab"
src="docs"                                  # folder to save (inside $lab)
dest="$lab/backup"
stamp=$(date +%Y%m%d_%H%M%S)                # e.g. 20261012_143005
archive="$dest/docs_$stamp.tar.gz"

if [ ! -d "$lab/$src" ]; then
```

```
    echo "Error: $lab/$src not found" >&2
    exit 1
fi
mkdir -p "$dest"                          # no error if it exists
if tar -czf "$archive" -C "$lab" "$src"; then
    n=$(tar -tzf "$archive" | grep -vc '/$')    # count files, not folders
    echo "Backup created: $archive"
    echo "Files: $n, size: $(du -h "$archive" | cut -f1)"
else
    echo "Error: tar failed" >&2
    exit 1
fi
```

Line by line:

- `$HOME/lab` rather than `~/lab`: a tilde is *not* expanded inside double quotes, `$HOME` is.
- `date +%Y%m%d_%H%M%S` produces a name such as `20261012_143005`: unique (to the second), and sortable, because the year comes first.
- `[ ! -d ... ]` stops early with a clear message if the source is missing.
- `mkdir -p` creates the destination if needed and is silent if it exists.
- `tar -czf archive -C dir docs` **c**reates a g**z**ip-compressed archive **f**ile; `-C` changes into `~/lab` first, so the paths inside the archive start with `docs/`. `if tar …` tests tar's own exit status.
- `tar -tzf` lists the archive; `grep -vc '/$'` counts the lines that do *not* end in `/`, i.e. files, not folders; `du -h | cut -f1` keeps only the size.

```
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085013.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls -l ../backup
total 4
-rw-rw-r-- 1 student student 862 Oct 12 08:50 docs_20261012_085013.tar.gz
student@okan-lab:~/lab/scripts$ tar -tzf ../backup/docs_*.tar.gz
docs/
docs/notes_02.txt
docs/syllabus.txt
docs/.settings
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab/scripts$ sleep 2
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085015.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls ../backup
docs_20261012_085013.tar.gz  docs_20261012_085015.tar.gz
```

Each run creates a new archive (the second run was two seconds later), so nothing is overwritten. The archive holds 7 files: the six visible documents plus the hidden `.settings` (`tar` includes hidden files). `final report.txt` is archived correctly because every variable is quoted. To restore into a scratch folder without touching your files: `tar -xzf ../backup/docs_<stamp>.tar.gz -C /tmp`. With `cron`, the

line `0 18 * * * $HOME/lab/scripts/backup.sh` in `crontab -e` would run the script every day at 18:00 (not exam material). Lab exercise 13 adds the deletion of old backups.

9.2 `report.sh`: summarising a log and a CSV file

```
#!/bin/bash
# report.sh -- one-page summary of logs/access.log and data/students.csv
log="$HOME/lab/logs/access.log"
csv="$HOME/lab/data/students.csv"
for f in "$log" "$csv"; do
    if [ ! -r "$f" ]; then echo "Error: cannot read $f" >&2; exit 1; fi
done
echo "===== Lab report, $(date '+%Y-%m-%d %H:%M') ====="

echo "--- Web server: $(basename "$log") ---"
total=$(wc -l < "$log")
errors=$(awk '$9 >= 400' "$log" | wc -l)    # field 9 = status code
echo "Requests: $total, errors (4xx/5xx): $errors ($((100 * errors / total))%)"
echo "Top 3 pages:"
awk '{print $7}' "$log" | sort | uniq -c | sort -rn | head -3

echo "--- Students: $(basename "$csv") ---"
echo "Students: $(tail -n +2 "$csv" | wc -l)"
echo "Per department:"
tail -n +2 "$csv" | cut -d, -f3 | sort | uniq -c
awk -F, 'NR > 1 { m += $4; f += $5; n++ }
         END { printf "Average midterm %.1f, average final %.1f\n", m/n, f/n }' "$csv"
passed=$(awk -F, 'NR > 1 && 0.4*$4 + 0.6*$5 >= 50' "$csv" | wc -l)
echo "Passed (0.4 x midterm + 0.6 x final >= 50): $passed"
```

```
student@okan-lab:~/lab/scripts$ ./report.sh
===== Lab report, 2026-10-12 08:50 =====
--- Web server: access.log ---
Requests: 300, errors (4xx/5xx): 98 (32%)
Top 3 pages:
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
--- Students: students.csv ---
Students: 40
Per department:
     10 CS
     17 ECT
      2 EE
     11 ME
Average midterm 67.0, average final 65.6
Passed (0.4 x midterm + 0.6 x final >= 50): 35
```

The script reuses the pipelines of Week 12: `awk '{print $7}'` extracts a field, `sort | uniq -c | sort -rn` counts and ranks, `tail -n +2` skips the CSV header and `cut -d, -f3` takes the department column. What is new is awk doing *decimal* arithmetic: `m += $4` adds up the midterm column, the `END` block runs once at the end, and `printf "%.1f"` prints one decimal. A condition alone, such as `0.4*$4 + 0.6*$5 >= 50`, is an awk *pattern*: awk prints only the lines for which it is true, and `wc -l` counts them. (The $40/60$ weighting is only an example.)

Checking the report

All numbers were recomputed independently in Python from the same files:

- 98 of 300 requests have a status ≥ 400: $100 \cdot 98/300 = 32.67$, and bash integer arithmetic prints 32.
- `/index.html` and `/images/logo.png` both have 33 requests; only one fits in the top 3, and `sort` breaks the tie.
- Departments: $17 + 11 + 10 + 2 = 40$ students.
- Average midterm 66.95 (printed rounded as 67.0), average final 65.6.
- 35 of the 40 students satisfy $0.4\,m + 0.6\,f \geq 50$.

10 Common Errors and What They Mean

The script `code/demo_errors.sh` creates five broken scripts. Their real error messages:

```
student@okan-lab:~/lab/scripts$ cat bad_space.sh
#!/bin/bash
name = "Ayse"
echo "Hello $name"
student@okan-lab:~/lab/scripts$ ./bad_space.sh
./bad_space.sh: line 2: name: command not found
Hello
student@okan-lab:~/lab/scripts$ cat bad_test.sh
#!/bin/bash
n=5
if [$n -gt 3]; then echo "big"; fi
student@okan-lab:~/lab/scripts$ ./bad_test.sh
./bad_test.sh: line 3: [5: command not found
student@okan-lab:~/lab/scripts$ cat bad_quotes.sh
#!/bin/bash
answer=""
if [ $answer = yes ]; then echo "OK"; fi
student@okan-lab:~/lab/scripts$ ./bad_quotes.sh
./bad_quotes.sh: line 3: [: =: unary operator expected
student@okan-lab:~/lab/scripts$ cat bad_if.sh
#!/bin/bash
if [ -d ~/lab ]; then
    echo "lab exists"
student@okan-lab:~/lab/scripts$ bash -n bad_if.sh
bad_if.sh: line 4: syntax error: unexpected end of file
```

Spaces around = `name: command not found`. Bash ran a command called `name`. Note that the script did *not* stop: it went on and printed `Hello` with an empty variable. Fix: `name="Ayse"`. The opposite mistake, `name= Ayse`, runs the command `Ayse` with an empty variable `name`.

No spaces inside [ ] `[5: command not found`. Fix: `if [ "$n" -gt 3 ]; then`.

Missing quotes `[: =: unary operator expected`. The empty, unquoted `$answer` vanished, leaving `[ = yes ]`. With quotes, `[ "" = yes ]` is simply false. Unquoted file names with spaces give `No such`

`file or directory` for each half (Section 4.3).

Missing `fi`, `done` or quote `syntax error: unexpected end of file`: a block was opened and never closed. `bash -n` finds this without running the script. The reported line is the *end* of the file; the real mistake is earlier.

Missing `chmod +x` `Permission denied`, exit status 126 (Section 3.2).

Not a number `integer expression expected` from `-lt`, `-gt`, … (Section 6.4).

10.1 Windows line endings (CRLF)

Windows editors end each line with two characters, carriage return and line feed (`\r\n`, "CRLF"); Linux uses only `\n`. A script written in Notepad and copied to Linux therefore has an invisible `\r` at the end of every line, which sticks to the last word of the line:

```
student@okan-lab:~/lab/scripts$ ./win.sh
bash: ./win.sh: cannot execute: required file not found
student@okan-lab:~/lab/scripts$ bash win.sh
win.sh: line 2: cd: $'/home/student/lab\r': No such file or directory
Hello from Windows
student@okan-lab:~/lab/scripts$ file win.sh
win.sh: Bourne-Again shell script, ASCII text executable, with CRLF line terminators
student@okan-lab:~/lab/scripts$ cat -A win.sh
#!/bin/bash^M$
cd ~/lab^M$
echo "Hello from Windows"^M$
student@okan-lab:~/lab/scripts$ sed -i 's/\r$//' win.sh
student@okan-lab:~/lab/scripts$ ./win.sh
Hello from Windows
```

- `./win.sh`: the kernel looks for an interpreter literally called `/bin/bash\r`, which does not exist. Bash 5.2 (used here) reports `cannot execute: required file not found`; older versions of bash (e.g. Ubuntu 22.04) report the classic `/bin/bash^M: bad interpreter: No such file or directory`. Both mean the same thing.
- `bash win.sh` runs, but `cd` receives the path with a `\r` at its end (shown as `$'…\r'`) and fails.
- `file` says `with CRLF line terminators`; `cat -A` shows each `\r` as `^M` before the end-of-line mark `$`.
- Fix: `sed -i 's/\r$//' win.sh` deletes the `\r` at the end of every line; `dos2unix win.sh` does the same if installed. In VS Code, click "CRLF" in the status bar and choose "LF".

11 Summary

Week 14 in eight sentences

1. A shell script is a text file of commands with `#!/bin/bash` on line 1; after `chmod +x` it runs with `./script.sh`, in a child process created by fork and exec.
2. Variables are assigned without spaces (`name=value`) and used in double quotes (`"$name"`); sin-

gle quotes make everything literal.

3. `$(command)` inserts a command's output, `$((expression))` computes with integers.
4. Arguments arrive as `$1` … `$9`, with `$#` and `"$@"`; `read -r` reads the keyboard.
5. Every command returns an exit status, 0 meaning success; `exit n` sets a script's status.
6. `[ ]` is a command that tests numbers (`-eq -lt -gt` …), strings (`= != -z`) and files (`-e -f -d`); `if / elif / else / fi` chooses a branch.
7. `for` loops over words, ranges and files; `while` repeats while a test succeeds or reads a file line by line; functions name a block of commands.
8. `bash -n` and `bash -x` help debug; quote your variables and save scripts with LF line endings.

12 Glossary

Argument (positional parameter) A word given after the script name; `$1`, `$2`, … inside the script.

Arithmetic expansion `$(( ))`: integer calculation by the shell.

Command substitution `$( )`: replaced by the output of the command inside.

CRLF Windows line ending `\r\n`; breaks scripts on Linux.

Exit status Number 0–255 returned by every command; 0 = success; in `$?`.

Function Named block of commands with its own arguments.

PATH Variable listing the folders where the shell looks for commands.

Quoting Double quotes keep spaces and expand variables; single quotes keep everything literal.

Shebang The first line `#!interpreter` that tells the kernel which program runs the script.

Shell script A text file of shell commands run by an interpreter such as bash.

test / [ ] Command that evaluates a condition and answers with its exit status.

13 Try It Yourself

Explore on your own machine

1. Put `cd /tmp` and `pwd` in a script and run it. In which folder is your terminal afterwards? Now run it with `source`. Explain the difference with Week 13 (processes).
2. Add `echo "My PID is $$, my parent is $PPID"` to `hello.sh`, run it twice, and compare with `echo $$` at the prompt.
3. Run `bash -x loops.sh`. How many times is the body of each loop executed?
4. Make `grade.sh` reject non-numeric input (use the `grep -qx` test from Section 6.4).
5. If you have `shellcheck`, run it on `quotes.sh` after deleting the two `# shellcheck` lines. What

does it report?

14 Review Questions

Conceptual

1. What does the line `#!/bin/bash` do, and who reads it?
2. Why does `./hello.sh` need the execute permission while `bash hello.sh` does not?
3. A script contains `cd /tmp`. After it ends, your terminal is still in your home folder. Why?
4. Explain the difference between `"$name"`, `'$name'` and `$name`.
5. Why must there be spaces inside `[ ]`, but no spaces around = in an assignment?
6. What exit status does a successful command return? What do 126 and 127 mean?
7. When is `read` the right tool, and when are arguments better?
8. What does `bad interpreter` (or `required file not found`) usually mean for a script copied from Windows, and how do you fix it?

Numerical and practical

9. What does `echo $(( (17 + 3) / 6 * 6 ))` print? And `echo $(( 17 % 5 ** 2 ))`?
10. `./args.sh "a b" c "" d`: what are `$#`, `$1` and `$3`?
11. A `while` loop starts with `n=10` and runs `n=$((n - 3))` while `[ "$n" -gt 0 ]`. How many times does the body run, and what is `n` at the end?
12. What do the three tests `[ 010 -eq 10 ]`, `[ 010 = 10 ]` and `[ -z "" ]` return (exit status)?
13. `for i in {1..4}; do s=$((s + i * i)); done` with `s=0`: what is `s`?
14. Write one line that prints `OK` if the folder `~/lab/backup` exists and `MISSING` otherwise.

Answers to the numerical and practical questions

9. $(17+3)/6 = 20/6 = 3$ (integer), $3 \times 6 = 18$. In the second, `**` (power) binds tighter than `%`: $5^2 = 25$ and $17 \bmod 25 = 17$.
10. `$#` $= 4$ (the empty string `""` is an argument too), `$1` $=$ `a b`, `$3` is empty.
11. n: $10 \to 7 \to 4 \to 1 \to -2$; the body runs 4 times and $n = -2$ at the end.
12. 0 (numerically equal), 1 (different text), 0 (the string is empty).
13. $s = 1 + 4 + 9 + 16 = 30$.
14. `[ -d ~/lab/backup ] && echo OK || echo MISSING` (or an `if/else/fi`).

15 Looking Ahead: Week 15

- **Week 15: Final exam review.** The final covers the whole course: the theory of Weeks 1–7 (use the Week 8 study guide) and the practical Linux weeks 9–14 — navigation, file operations, permissions, text processing and pipelines, processes and job control, and shell scripting.
- Expect "write the command / pipeline / script" questions: the Lab Sheet above is the best preparation.
- The exam date, time and room will be announced on the course Telegram channel.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Section 2.2 (user and operating-system interface: command interpreters, shells) and Section 3.3 (process creation with `fork()` and `exec()`).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Chapter 5, "Interlude: Process API" (how a shell uses fork, exec and wait): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
- GNU Bash Reference Manual: `https://www.gnu.org/software/bash/manual/bash.html`
- W. Shotts, *The Linux Command Line*, Part 4 "Writing Shell Scripts" (free PDF): `https://linuxcommand.org/tlcl.php`
- ShellCheck, finds bugs in shell scripts: `https://www.shellcheck.net`

16 Lab Sheet: Graded Script-Writing Exercises

Write each script in `~/lab/scripts`, starting from a fresh lab folder (`bash ~/mcmp117_lab_setup.sh`). Every script must start with `#!/bin/bash`, have a comment line saying what it does, be executable, pass `bash -n`, and quote its variables. Name them `exNN_name.sh` as shown. In class, exercises 2, 6, 11 and 13 are done and graded (slide "In-Class Lab"); the rest are homework and exam practice. Total: 100 points. Suggested rubric for each exercise: runs without errors 40%, correct output 30%, handles bad input 15%, style (comments, quoting, shebang) 15%.

Part A --- Basics (5 points each)

1. **`ex01_myinfo.sh`** Print your user name, your home folder, today's date as `YYYY-MM-DD`, your login shell, and the number of students in `~/lab/data/students.csv` (lines minus the header).
2. **`ex02_power.sh N`** Print N, N^2 and N^3. Without exactly one argument print a Usage line to stderr and exit with status 2.
3. **`ex03_hello_name.sh NAME [CITY]`** Print `Hello NAME from CITY!`; if no city is given, use Istanbul; if no name is given, print a Usage line and exit 1.
4. **`ex04_calc.sh`** Read two integers with `read` and print their sum, difference, product, and integer quotient with remainder. Refuse to divide by zero.

Part B --- Decisions and simple loops (6 points each)

5. **`ex05_sign.sh N`** Say whether N is positive, negative or zero, and whether it is even or odd.
6. **`ex06_what.sh PATH…`** For every argument print *file* (with its line count), *directory* or *missing*. Exit with status 1 if at least one path is missing, else 0.
7. **`ex07_table.sh [N]`** Print the multiplication table $N \times 1 \ldots N \times 10$ (default $N = 7$) with a `for` loop.
8. **`ex08_sum.sh [N]`** Compute $1 + 2 + \cdots + N$ with a `while` loop (default $N = 100$) and compare with the formula $\frac{N(N+1)}{2}$.

Part C --- Files and data (7 points each)

9. **`ex09_doclines.sh`** For every `.txt` file in `~/lab/docs`, print its name and number of lines; finally print the total and the longest file.
10. **`ex10_minmax.sh`** Read `~/lab/data/numbers.txt` with a `while read` loop and print the count, minimum, maximum, sum and integer average — without `sort`.
11. **`ex11_levels.sh`** Count the `INFO`, `WARN` and `ERROR` lines of `~/lab/logs/system.log` with a `for` loop and `grep -c`, and print each count as a percentage of all lines.
12. **`ex12_temp.sh`** Write a function `c2f` that converts Celsius to Fahrenheit, $F = \frac{9}{5}C + 32$, and print a table for $-10, 0, 10, \ldots, 40$ °C.

Part D --- Challenge (7 points each)

13. **`ex13_backup_keep.sh`** Extend `backup.sh` so that only the 3 newest archives in `~/lab/backup` are kept; print the name of every archive it deletes. Test it by running it five times, one second apart.
14. **`ex14_guess.sh [SECRET]`** A guessing game: the secret is a number from 1 to 100 (the argument, or random with `$RANDOM`); read guesses until the right one, answer *Too small* / *Too big*, and finally print the number of tries.
15. **`ex15_dept.sh DEPT`** Print the number of students and the average final grade (one decimal) of one department from `students.csv`; exit 1 with a message for an unknown department.
16. **`ex16_status_bars.sh`** For every HTTP status code in `access.log` print the code, its count and a bar with one `#` per 5 requests (rounded down).

Points: $4 \times 5 + 4 \times 6 + 8 \times 7 = 20 + 24 + 56 = 100$.

17 Lab Sheet Solutions

These solutions are in `code/solutions/`. Each was run with bash 5.2 in a fresh lab folder and passes `bash -n` and ShellCheck; the outputs are real (file `code/solutions/out_solutions.txt`). Other correct solutions are of course possible. Try first, then compare!

1. ex01_myinfo.sh

```
#!/bin/bash
# ex01_myinfo.sh -- Lab exercise 1: information about me and my lab
echo "User:      $(whoami)"
echo "Home:      $HOME"
echo "Date:      $(date +%Y-%m-%d)"
echo "Shell:     $SHELL"
students=$(( $(wc -l < ~/lab/data/students.csv) - 1 ))   # minus the header line
echo "Students in students.csv: $students"
```

```
student@okan-lab:~/lab/scripts$ ./ex01_myinfo.sh
User:      student
Home:      /home/student
Date:      2026-10-12
Shell:     /bin/bash
Students in students.csv: 40
```

2. ex02_power.sh

```
#!/bin/bash
# ex02_power.sh -- Lab exercise 2: square and cube of a number
if [ $# -ne 1 ]; then
    echo "Usage: $0 NUMBER" >&2
    exit 2
fi
n=$1
echo "n = $n, n^2 = $((n * n)), n^3 = $((n * n * n))"
```

```
student@okan-lab:~/lab/scripts$ ./ex02_power.sh 12
n = 12, n^2 = 144, n^3 = 1728
student@okan-lab:~/lab/scripts$ ./ex02_power.sh
Usage: ./ex02_power.sh NUMBER
```

3. ex03_hello_name.sh

```
#!/bin/bash
# ex03_hello_name.sh -- Lab exercise 3: greeting with a required and an optional argument
if [ $# -lt 1 ]; then
    echo "Usage: $0 NAME [CITY]" >&2
    exit 1
fi
city=$2
if [ -z "$city" ]; then
    city="Istanbul"                 # default when no city is given
fi
echo "Hello $1 from $city!"
```

```
student@okan-lab:~/lab/scripts$ ./ex03_hello_name.sh Elif
Hello Elif from Istanbul!
student@okan-lab:~/lab/scripts$ ./ex03_hello_name.sh Can Izmir
Hello Can from Izmir!
```

4. ex04_calc.sh

```
#!/bin/bash
# ex04_calc.sh -- Lab exercise 4: read two integers and calculate
read -r -p "First integer:  " a
read -r -p "Second integer: " b
echo "$a + $b = $((a + b))"
echo "$a - $b = $((a - b))"
echo "$a * $b = $((a * b))"
if [ "$b" -eq 0 ]; then
    echo "Cannot divide by zero"
else
```

```
    echo "$a / $b = $((a / b)) remainder $((a % b))"
fi
```

```
student@okan-lab:~/lab/scripts$ ./ex04_calc.sh
First integer:  17
Second integer: 5
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3 remainder 2
```

5. ex05_sign.sh

```
#!/bin/bash
# ex05_sign.sh -- Lab exercise 5: positive / negative / zero, and even / odd
if [ $# -ne 1 ]; then
    echo "Usage: $0 INTEGER" >&2
    exit 2
fi
n=$1
if [ "$n" -gt 0 ]; then
    sign="positive"
elif [ "$n" -lt 0 ]; then
    sign="negative"
else
    sign="zero"
fi
if [ $((n % 2)) -eq 0 ]; then parity="even"; else parity="odd"; fi
echo "$n is $sign and $parity"
```

```
student@okan-lab:~/lab/scripts$ ./ex05_sign.sh -8
-8 is negative and even
student@okan-lab:~/lab/scripts$ ./ex05_sign.sh 0
0 is zero and even
student@okan-lab:~/lab/scripts$ ./ex05_sign.sh
Usage: ./ex05_sign.sh INTEGER
```

6. ex06_what.sh

```
#!/bin/bash
# ex06_what.sh -- Lab exercise 6: classify every argument; exit 1 if one is missing
status=0
for p in "$@"; do
    if [ -f "$p" ]; then
        echo "$p: file ($(wc -l < "$p") lines)"
    elif [ -d "$p" ]; then
        echo "$p: directory"
    else
        echo "$p: missing"
        status=1
    fi
done
exit $status
```

```
student@okan-lab:~/lab/scripts$ ./ex06_what.sh ../docs/syllabus.txt ../src nothing.txt
../docs/syllabus.txt: file (15 lines)
../src: directory
nothing.txt: missing
student@okan-lab:~/lab/scripts$ echo $?
1
```

The variable `status` remembers a failure while the loop continues with the other arguments.

7. ex07_table.sh

```bash
#!/bin/bash
# ex07_table.sh -- Lab exercise 7: multiplication table of N
n=${1:-7}                          # default 7 when no argument is given
for i in {1..10}; do
    echo "$n x $i = $((n * i))"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex07_table.sh 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

8. ex08_sum.sh

```bash
#!/bin/bash
# ex08_sum.sh -- Lab exercise 8: 1 + 2 + ... + N with a while loop
n=${1:-100}
i=1
sum=0
while [ "$i" -le "$n" ]; do
    sum=$((sum + i))
    i=$((i + 1))
done
echo "Loop:    1 + ... + $n = $sum"
echo "Formula: n(n+1)/2   = $((n * (n + 1) / 2))"
```

```
student@okan-lab:~/lab/scripts$ ./ex08_sum.sh 100
Loop:    1 + ... + 100 = 5050
Formula: n(n+1)/2   = 5050
```

Check: $\frac{100 \cdot 101}{2} = 5050$.

9. ex09_doclines.sh

```bash
#!/bin/bash
# ex09_doclines.sh -- Lab exercise 9: lines per document, total, longest
total=0
max=0
longest=""
for f in ~/lab/docs/*.txt; do
    n=$(wc -l < "$f")
    printf '%-18s %3d\n' "$(basename "$f")" "$n"
    total=$((total + n))
    if [ "$n" -gt "$max" ]; then
        max=$n
        longest=$(basename "$f")
    fi
done
echo "Total: $total lines; longest: $longest ($max lines)"
```

```
student@okan-lab:~/lab/scripts$ ./ex09_doclines.sh
final report.txt     2
linux_quotes.txt     5
notes_01.txt         1
notes_02.txt         1
notes_03.txt         1
```

```
syllabus.txt        15
Total: 25 lines; longest: syllabus.txt (15 lines)
```

`printf '%-18s %3d'` prints the name left-aligned in 18 columns and the number right-aligned in 3. Total: $2 + 5 + 1 + 1 + 1 + 15 = 25$.

10. ex10_minmax.sh

```
#!/bin/bash
# ex10_minmax.sh -- Lab exercise 10: minimum, maximum, sum, average of numbers.txt
file=~/lab/data/numbers.txt
count=0
sum=0
while read -r x; do
    if [ "$count" -eq 0 ]; then min=$x; max=$x; fi   # first number
    if [ "$x" -lt "$min" ]; then min=$x; fi
    if [ "$x" -gt "$max" ]; then max=$x; fi
    sum=$((sum + x))
    count=$((count + 1))
done < "$file"
echo "count=$count min=$min max=$max sum=$sum average=$((sum / count))"
```

```
student@okan-lab:~/lab/scripts$ ./ex10_minmax.sh
count=20 min=280 max=998 sum=11325 average=566
```

The first number initialises both `min` and `max`. Same sum as `while.sh`: $11\,325$, average 566 (integer).

11. ex11_levels.sh

```
#!/bin/bash
# ex11_levels.sh -- Lab exercise 11: messages per level in system.log
log=~/lab/logs/system.log
total=$(wc -l < "$log")
for level in INFO WARN ERROR; do
    n=$(grep -c " $level " "$log")
    echo "$level: $n of $total lines ($((100 * n / total))%)"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex11_levels.sh
INFO: 79 of 120 lines (65%)
WARN: 28 of 120 lines (23%)
ERROR: 13 of 120 lines (10%)
```

$79 + 28 + 13 = 120$. The percentages add up to 98%, not 100%, because integer division truncates ($65.8 \to 65$, $23.3 \to 23$, $10.8 \to 10$). The spaces in `" $level "` make sure only the level column matches.

12. ex12_temp.sh

```
#!/bin/bash
# ex12_temp.sh -- Lab exercise 12: a function that converts Celsius to Fahrenheit
c2f() {
    echo $(( $1 * 9 / 5 + 32 ))        # exact for multiples of 5
}
echo "Celsius  Fahrenheit"
for c in -10 0 10 20 30 40; do
    printf '%7d  %10d\n' "$c" "$(c2f "$c")"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex12_temp.sh
Celsius  Fahrenheit
    -10          14
      0          32
```

```
     10          50
     20          68
     30          86
     40         104
```

For multiples of 5, $9C/5$ is an integer, so the integer result is exact: $40 \cdot 9/5 + 32 = 104$.

13. ex13_backup_keep.sh

```bash
#!/bin/bash
# ex13_backup_keep.sh -- Lab exercise 13: backup.sh that keeps only the 3 newest archives
lab="$HOME/lab"
dest="$lab/backup"
keep=3
archive="$dest/docs_$(date +%Y%m%d_%H%M%S).tar.gz"
mkdir -p "$dest"
tar -czf "$archive" -C "$lab" docs || exit 1
echo "Created $(basename "$archive")"
# names contain only letters, digits and _ , so ls is safe here; newest first
# shellcheck disable=SC2012
ls -1t "$dest"/docs_*.tar.gz | tail -n +$((keep + 1)) | while read -r old; do
    echo "Removing old backup $(basename "$old")"
    rm -- "$old"
done
```

```
student@okan-lab:~/lab/scripts$ for _ in 1 2 3 4 5; do ./ex13_backup_keep.sh; sleep 1; done
Created docs_20261012_085022.tar.gz
Created docs_20261012_085023.tar.gz
Created docs_20261012_085024.tar.gz
Created docs_20261012_085025.tar.gz
Removing old backup docs_20261012_085022.tar.gz
Created docs_20261012_085026.tar.gz
Removing old backup docs_20261012_085023.tar.gz
student@okan-lab:~/lab/scripts$ ls ../backup
docs_20261012_085024.tar.gz  docs_20261012_085025.tar.gz  docs_20261012_085026.tar.gz
```

`ls -1t` lists the archives newest first; `tail -n +4` skips the three newest and passes the rest to the loop. **Careful:** `rm` has no recycle bin (Week 10). Test such a script on copies first, and print what you delete. Parsing `ls` output is safe here only because the names contain no spaces.

14. ex14_guess.sh

```bash
#!/bin/bash
# ex14_guess.sh -- Lab exercise 14: guess the secret number (1-100)
secret=${1:-$((RANDOM % 100 + 1))}       # fixed secret if given (for testing)
tries=0
guess=0
while [ "$guess" -ne "$secret" ]; do
    read -r -p "Your guess: " guess
    tries=$((tries + 1))
    if [ "$guess" -lt "$secret" ]; then
        echo "Too small"
    elif [ "$guess" -gt "$secret" ]; then
        echo "Too big"
    fi
done
echo "Correct! You needed $tries tries."
```

```
student@okan-lab:~/lab/scripts$ ./ex14_guess.sh 37
Your guess: 50
Too big
Your guess: 25
Too small
Your guess: 37
```

```
Correct! You needed 3 tries.
```

The player typed 50, 25 and 37. With a fixed secret the game is testable; without an argument, `$RANDOM % 100 + 1` gives 1–100.

15. ex15_dept.sh

```
#!/bin/bash
# ex15_dept.sh -- Lab exercise 15: students and average final of one department
csv=~/lab/data/students.csv
dept=$1
if [ -z "$dept" ]; then
    echo "Usage: $0 DEPARTMENT" >&2
    exit 2
fi
n=$(cut -d, -f3 "$csv" | grep -cx "$dept")
if [ "$n" -eq 0 ]; then
    echo "No students in department $dept" >&2
    exit 1
fi
avg=$(awk -F, -v d="$dept" '$3 == d { s += $5; k++ } END { printf "%.1f", s/k }' "$csv")
echo "$dept: $n students, average final $avg"
```

```
student@okan-lab:~/lab/scripts$ ./ex15_dept.sh ECT
ECT: 17 students, average final 63.1
student@okan-lab:~/lab/scripts$ ./ex15_dept.sh EE
EE: 2 students, average final 63.0
student@okan-lab:~/lab/scripts$ ./ex15_dept.sh XYZ
No students in department XYZ
```

`grep -cx` counts lines that match the *whole* department name; `awk -v d=…` passes the shell variable into awk. Check: ECT finals sum to 1072, $1072/17 = 63.06$.

16. ex16_status_bars.sh

```
#!/bin/bash
# ex16_status_bars.sh -- Lab exercise 16: bar chart of HTTP status codes (# = 5 requests)
log=~/lab/logs/access.log
awk '{print $9}' "$log" | sort | uniq -c | while read -r count code; do
    bar=""
    for ((i = 0; i < count / 5; i++)); do bar="$bar#"; done
    printf '%s %4d %s\n' "$code" "$count" "$bar"
done
```

```
student@okan-lab:~/lab/scripts$ ./ex16_status_bars.sh
200  148 ############################
301   20 ####
304   34 ######
403   17 ###
404   57 ###########
500   24 ####
```

`while read -r count code` splits each line of `uniq -c` into two variables. Check: $148/5 = 29.6 \to 29$ characters; the counts sum to 300.